

OWASP Juice Shop — Desktop Security Test Report

VERDICT · September 2026 · Assessment `a-mtoijulk-ajc469`

Executive summary

I assessed OWASP Juice Shop 20.2.0 with VERDICT Desktop using **OpenCodeGo / omen-alpha**. I started the scan at `http://localhost:3000/` and used one administrator account and two customer accounts to test unauthenticated access and access across account boundaries.

VERDICT mapped and tested **154 screens**. The saved model counter recorded **115,116,331 tokens**, and I paid **US\$9.92** for the run. The saved tests include successful login SQL injection, browser-executed DOM XSS, cross-user basket access, and an address update that transferred another customer's record to the caller.

I organized the results into **23 test cases** to group repeated observations of the same behavior. T01–T18 cover the observed results. T19–T23 cover cases I have left open for an access-policy decision or further technical verification. I state the supporting evidence and remaining questions in each case.

[Scope and configuration](#) · [Test results](#) · [Remediation plan](#) · [Result traceability](#) · [HTML](#) · [PDF](#)

Scope and configuration

Setting	Value
Target	OWASP Juice Shop 20.2.0, local Docker deployment
Entry URL	<code>http://localhost:3000/</code>
Assessment interface	VERDICT desktop application
Provider	OpenCodeGo, OpenAI-compatible API
Model	<code>omen-alpha</code>
API base URL	<code>https://opencode.ai/zen/go/v1</code>
Accounts	<code>admin</code> : administrator; <code>user1</code> and <code>user2</code> : customers
Exploration scope	Same origin, <code>localhost:3000</code> , path prefix <code>/</code>
Maximum crawl depth	10
Stored scope rate policy	30 requests/minute; maximum 2 concurrent requests
Traffic proxy	Burp proxy at <code>http://127.0.0.1:8080</code>
Start	September 6, 2026, 00:04 JST
Original report export	September 7, 2026, 08:54 JST
Final recorded phase	Report

The timestamps cover the run and export window, including interruptions; they are not a measurement of active scanning time. The scope rate values are the saved policy, not a measured request-rate average.

Model consumption and cost

Measure	Result
Recorded model tokens	115,116,331 — approximately 115.12 million
LLM usage charge for the assessment	US\$9.92
Model used	OpenCodeGo / omen-alpha

I took the token figure from VERDICT's saved assessment counter and paid US\$9.92 for this run. The saved counter has no billed input/output/cache breakdown, so I report token consumption and the actual charge separately.

Test method and execution limits

I combined application exploration, direct HTTP probes, browser execution checks, and comparisons between administrator, customer, and unauthenticated sessions. For replay-based findings, a useful control must differ from the attack in the tested condition, and successful attack responses must be reproducible. I identify cases where the saved evidence does not establish those conditions below.

The following limits affected this run:

- **Burp active scanning did not run.** Two submission attempts were rejected with HTTP 401, `invalid or missing X-Scan-Token` ; both ended with nothing submitted. Burp proxy use should not be read as successful Burp Scanner coverage.
- **Blind XXE remained unresolved.** XML acceptance produced no file-content or out-of-band evidence of entity expansion.
- **Upload acceptance did not establish stored XSS.** Script-bearing uploads were stored, but browser execution was not demonstrated.
- **154 tested screens is inventory coverage.** It does not mean every vulnerability class was exercised on every screen.

Test results

Coverage and recorded output

Measure	Result
Mapped screens	154
Screens marked tested	154
Excluded / unfinished screens	0 / 0
Saved reviewed output	54 entries in the confirmed category; 5 suspected leads; 1 low-signal note
Presentation in this report	18 observed-result cases and 5 follow-up cases

The saved severity totals were 3 critical, 22 high, 15 medium, 12 low, and 2 informational entries in the confirmed category. I grouped overlapping entries from these engine classifications into the cases below and narrowed conclusions to what the saved evidence supports.

T01 — SQL injection in the login endpoint

Result: authentication bypass observed. A request without an existing session obtained an administrator session through a boolean SQL injection in the login email field.

Test	Recorded response
POST /rest/user/login with email ' OR '1'='2'-- - and dummy password x	HTTP 401, invalid credentials
Same request with email ' OR '1'='1'-- -	HTTP 200 and an administrator authentication token
Repeated true-condition request	HTTP 200 and an administrator session again

Impact. An unauthenticated caller can bypass the password check and assume the identity selected by the injected query. The test establishes session issuance; individual administrator functions require their own authorization checks.

Recommendation. Parameterize the login query, retrieve a single account by a validated identifier, and verify its password using the password-verification function. Add regression cases for both boolean payloads and ordinary failed logins.

Evidence: ev-mtp3sqqv-1jx , ev-mtp3sqz9-1jy , ev-mtp3sr7a-1jz . Source entry: 2.

T02 — SQL injection in product search

Result: cross-table data retrieval observed. The q parameter of GET /rest/products/search accepted a UNION query. The response included user-table email and password-hash fields within the product response structure.

Test. A normal search returned product records. A nine-column UNION selecting user fields returned account data without authentication and was replayed successfully. A mismatched column count produced a SQLite UNION error. A related query also retrieved a soft-deleted product that the ordinary search did not return.

Impact. The search endpoint exposed records outside its intended product result set, including credential material. The soft-deleted product observation is another use of this injection point, rather than a separate SQL injection. Password recovery from the exposed hashes was not demonstrated.

Recommendation. Bind search parameters, enforce the product visibility predicate server-side, and restrict the database account to the tables and operations needed by the application.

Evidence: `ev-mtoodlfj-d8` , `ev-mtooeqvj-dd` , `ev-mtoofilx-dg` ; hidden-product query: `ev-mtp0xujw-1de` . Source entries: 1, 46.

T03 — DOM XSS in the search route

Result: script execution observed in the browser. The client-side route `/#/search?q=` rendered attacker-controlled markup into an executable DOM context.

Test. A benign search marker did not execute script. An image payload with an `onerror` handler set `window.__verdict_xss` in the browser; a second navigation reproduced execution. The HTTP response was the SPA shell, so server-response reflection alone did not establish this issue. Repeated discoveries of the same search sink are grouped here.

Impact. An attacker-controlled search URL can execute script in the application's origin. The recorded probe demonstrates execution, not theft of a session or completion of a privileged action.

Recommendation. Render search input as text. If rich HTML is required, sanitize it with an explicit allowlist before insertion. Add a browser regression test that asserts the execution marker remains absent, and deploy CSP as defense in depth.

Evidence: `ev-mtojgson-x` , `ev-mtojgtf6-y` , `ev-mtojgu5b-z` . Source entry: 26.

T04 — Acceptance of unsigned JWTs

Result: forged-token acceptance observed. Protected endpoints accepted an attacker-constructed JWT with `alg: none` .

Test. A garbage bearer token received HTTP 401 at `/rest/wallet/balance` . An unsigned token carrying chosen identity claims received HTTP 200 on repeat requests. An unsigned token claiming administrator identity also received the administrator user record from `/api/Users/1` .

Impact. Signature verification could be bypassed for the tested routes, allowing caller-controlled identity claims to be trusted. The user-record read does not by itself prove that every administrator-only operation is available.

Recommendation. Require signed tokens, configure an explicit algorithm allowlist, and validate issuer, audience, expiry, and signature consistently across routes. Derive authorization decisions from trusted server-side account state.

Evidence: `ev-mtpbhléo-27p` , `ev-mtpbhlnd-27q` , `ev-mtpbhlvr-27r` ; administrator-record reads: `ev-mtpbk7z4-27x` , `ev-mtpbk87n-27y` . Source entry: 23.

T05 — Cross-user basket and basket-item reads

Result: object ownership checks were bypassed on reads. A customer session retrieved basket data belonging to another customer.

Test. At `/rest/basket/{id}`, the caller's own basket provided a baseline; a foreign basket returned its contents, while a nonexistent basket returned `data: null`. An unauthenticated request was rejected with HTTP 401. At `/api/BasketItems/{id}`, a foreign item was returned with its foreign `BasketId`, while a nonexistent item received HTTP 404.

Impact. Authentication was present, but the tested reads exposed another customer's product selections and quantities. A nonexistent-object control establishes routing behavior; the separate owner IDs establish the cross-user access.

Recommendation. Resolve baskets through the authenticated account and authorize basket-item access through the parent basket. Test own, foreign, and nonexistent objects with at least two customer accounts.

Evidence: `ev-mtojev0i-p`, `ev-mtojevi9-r`, `ev-mtojfetx-s`; item reads: `ev-mtozmsa9-18x`, `ev-mtozmsrc-18z`. Source entries: 5, 12.

T06 — Cross-user address update and ownership transfer

Result: unauthorized modification and ownership transfer observed. The address API applied different ownership rules to reads and updates.

Test. With the `user2` session, a foreign address was absent from the caller's address list and a direct `GET /api/Addresss/{id}` returned HTTP 400. A `PUT` to that same ID, using JSON content type and a changed `city`, returned HTTP 200. The response showed the owner changed from the other customer to the caller, and a subsequent address-list request included the transferred record.

Impact. A customer could modify and take ownership of an existing foreign address. The follow-up list corroborates persistence; it is not a second independent write replay.

Recommendation. Apply the same owner-scoped lookup to every method. Keep ownership fields immutable through customer update requests, and verify the record's original owner before applying any change.

Evidence: `ev-mtphtc63-2vj`, `ev-mtphxzmu-2vu`, `ev-mtphyow7-2vx`. Source entry: 24.

T07 — Recycling records: unscoped reads and supplied ownership

Result: foreign-record reads and owner-field assignment observed. The recycling API exposed records without a session and accepted a client-selected owner when creating a record.

Test. An unauthenticated `GET /api/Recycles/{id}` returned a record belonging to another user; a nonexistent ID returned an empty result. During creation, a request without `UserId` stored a null owner, while a customer request containing another user's `UserId` stored that supplied value. A follow-up read returned the created record.

Impact. The tested API allowed disclosure of recycling-request data and creation of records attributed to another account. The read and create behaviors need separate access checks even though they concern the same resource.

Recommendation. Require authentication where recycling data is private, scope reads to the owner, and assign ownership from the session during creation. Reject unexpected ownership fields.

Evidence: `ev-mtozc041-18d`, `ev-mtozar1d-18b`, `ev-mtoze2wy-18j`; creation: `ev-mtoz8f06-181`, `ev-mtoz8f9b-182`. Source entries: 10, 11.

T08 — Product writes and unrestricted metadata assignment

Result: unauthorized catalog changes observed. Customer sessions created products and changed product prices. A separate test updated a product without a session.

Test. Customer `POST /api/Products` returned HTTP 201, and a price update returned the supplied value. With credentials removed, `PUT /api/Products/{id}` still returned a modified record, while unauthenticated create/delete controls were rejected. Creation also accepted a supplied `createdAt` ; a supplied `deletedAt` led to an error and a record that could not subsequently be fetched.

Impact. The tested write paths permitted catalog tampering. Accepted timestamps affect record integrity. The `deletedAt` error and failed fetch do not, without a database check, establish the exact state of a hidden database row.

Recommendation. Require an authorized catalog-management role for each write method. Allowlist customer-editable fields, keep timestamps and deletion state server-controlled, and validate updates before persistence.

Evidence: `ev-mtp58vog-1mo` , `ev-mtp55zgw-1ml` ; unauthenticated update: `ev-mtp6em8w-1qu` , `ev-mtp6et1i-1qv` , `ev-mtp6f4pg-1qw` . Source entries: 18, 19, 20.

T09 — Order tracking and invoice access

Result: order data and invoice retrieval observed without owner verification. Order codes created under the two customer accounts were used to test access from another account and from an empty session.

Test. `/rest/track-order/{code}` returned order details for a valid foreign code. An invalid code returned only an echo, without order data. The same valid order details were available without authentication. Corresponding invoice PDFs under `/ftp/order_{code}.pdf` were also downloadable without a session; a nonexistent invoice returned HTTP 404.

Impact. Anyone possessing a tested order identifier could retrieve the associated details and invoice. These requests do not establish how easily an attacker can discover an unknown code, or that its effective entropy is limited to its four-character prefix.

Recommendation. Authenticate and authorize order access. If shareable bearer links are intentional, use sufficiently random, scoped, expiring identifiers and limit the data returned through them. Apply the same policy to invoice storage.

Evidence: `ev-mtp3mew7-1jk` , `ev-mtp3mf4f-1jl` , `ev-mtp3nh03-1jn` ; invoices: `ev-mtp52eyz-1md` , `ev-mtp52eqv-1mc` . Source entries: 15, 16, 17.

T10 — Excessive account fields in API responses and tokens

Result: account data beyond the requesting workflow was returned. Several routes serialized sensitive account fields.

Route or artifact	Recorded behavior
<code>/rest/user/whoami?fields=password</code>	Returned the current caller's password digest when requested by field name
Session JWT	Included a password digest in readable claims

Route or artifact	Recorded behavior
<code>/rest/user/authentication-details</code>	A customer session received a user roster with account and credential-related fields
<code>/api/Users/</code>	A customer session received other users' email, role, and membership-token fields
<code>/rest/memories</code>	An unauthenticated response included related user objects containing password digests and other account data

Impact. The exposure differs by route: a caller's own digest, another user's profile, and an unauthenticated cross-user digest are not equivalent. Hash disclosure enables offline guessing but does not prove plaintext-password recovery. A sensitive field name alone also does not establish that its value is populated.

Recommendation. Use explicit response schemas for each workflow. Exclude password digests, MFA secrets, and membership credentials from client responses and JWT claims. Restrict administrative account listings and reassess exposed credentials.

Evidence: `ev-mtowzryl-y7` , `ev-mtowzs6o-y8` , `ev-mtp1fqqr-1f0` , `ev-mtovadas-up` , `ev-mtozsowh-19n` .
Source entries: 9, 13, 32, 34, 35.

T11 — Public file exposure and extension-filter bypass

Result: restricted-extension files and sensitive documents were downloadable. The `/ftp` listing exposed backup and support filenames, and encoded suffixes bypassed the file-extension check.

Test. A direct request for `/ftp/package.json.bak` received HTTP 403. Appending `%2500.md` returned the backup content on repeated requests. Related tests retrieved a coupon backup, a KeePass database, and documents through the same directory. The acquisition document was directly readable; the quarantine subdirectory exposed listings and shortcut files.

Impact. The tested paths disclosed backup material and documents. Retrieving a KeePass file does not establish recovery of its contents; retrieving historical coupon strings does not prove that a current purchase accepts them. Quarantine shortcut retrieval did not demonstrate execution of the referenced binaries. These observations share the file-publication and path-validation problem and are grouped together.

Recommendation. Keep backups, support vaults, invoices, and internal documents outside the web root. Normalize paths before checking them, reject embedded null bytes, use explicit file mappings, and enforce authorization at download time.

Evidence: `ev-mtowf5mv-wt` , `ev-mtowf5uw-wu` , `ev-mtowf62x-wv` , `ev-mtowf6j6-wx` , `ev-mtpkumls-38p` .
Source entries: 8, 21, 25, 33, 37, 39, 56.

T12 — Negative basket quantities

Result: an invalid quantity changed the stored basket calculation. `PUT /api/BasketItems/{id}` accepted `quantity: -1` .

Test. A large positive quantity was rejected by the stock check. The negative value was accepted, and a subsequent basket read showed the negative item quantity and a lower calculated total. Setting the quantity back to a positive value restored the baseline behavior.

Impact. The basket accepted an invalid business value that reduced its total. The recorded basket read does not, by itself, prove fulfillment of goods or a successful external payment at that price.

Recommendation. Require positive integer quantities within stock and purchase limits. Recompute and validate totals at checkout using server-controlled prices and quantities.

Evidence: `ev-mtpac52j-235` , `ev-mtpabzlo-233` , `ev-mtpac1wn-234` . Source entry: 22.

T13 — Negative wallet deposits

Result: negative top-up amounts were accepted. The wallet deposit endpoint accepted a negative `balance` value with a payment-method reference.

Test. A positive deposit returned a positive amount. Repeated negative-deposit requests returned the supplied negative amount. The saved reproduction also records a subsequent balance read after a larger negative deposit.

Impact. The API did not enforce a positive deposit amount. This establishes an amount-validation problem; it does not establish an external payout, theft, or payment-processor bypass. Financial impact requires reconciliation against the wallet ledger and payment records.

Recommendation. Reject nonpositive and invalid amounts before initiating payment or updating the ledger. Use atomic ledger entries and reconcile every credited amount with a successful payment event.

Evidence: `ev-mtp0hlou-1c6` , `ev-mtp0hlxw-1c7` , `ev-mtp0hm6n-1c8` . Source entry: 36.

T14 — Feedback ownership, CAPTCHA, and input validation

Result: forged attribution and invalid feedback values were persisted. The feedback workflow accepted caller-selected identity fields and did not enforce its UI constraints on the server.

Test. A feedback submission without `UserId` stored a null owner; supplying another user's ID stored that value. Ratings outside the UI's 1–5 range and a missing comment were accepted. Separately, `/rest/captcha` returned the answer with the challenge, and the same challenge/answer pair could be reused. An incorrect answer was rejected, showing that a check existed but was ineffective against a client that retrieved the answer.

Impact. Clients could submit feedback attributed to another account, poison rating data, and automate submissions past the tested CAPTCHA. These observations do not measure the application's overall abuse capacity or all rate limits.

Recommendation. Derive authorship from the session, validate the complete feedback schema, keep CAPTCHA answers server-side, expire challenges after one use, and add rate limits appropriate to the workflow.

Evidence: `ev-mtou9civ-sd` , `ev-mtou9anz-sc` , `ev-mtouaokt-sg` , `ev-mtou7ldk-rx` , `ev-mtou7gaz-rv` . Source entries: 7, 31, 43.

T15 — Redirect allowlist bypass

Result: an external destination passed the redirect check. `/redirect?to=` accepted a destination on another host when an allowed URL appeared inside its query string.

Test. An ordinary allowed URL redirected as expected. An unapproved destination was rejected with HTTP 406. A value such as `https://veritas-oob.example?https://github.com/juice-shop/juice-shop` received HTTP 302 with the external host in `Location` , and the behavior was repeated.

Impact. A link through the application's redirect endpoint can lead to an attacker-selected site. The observed response establishes an open redirect; credential capture or phishing success was not tested.

Recommendation. Parse the destination as a URL, compare its normalized scheme and host against an exact allowlist, and reject ambiguous or invalid destinations.

Evidence: `ev-mtpcrioz-2b6` , `ev-mtpcrix1-2b7` , `ev-mtpcrj52-2b8` . Source entry: 38.

T16 — Account enumeration and short login-burst behavior

Result: account existence was distinguishable; no throttling was observed within the tested burst. The security-question endpoint and login endpoint returned different useful signals.

Test. `/rest/user/security-question?email=` returned a question for a registered test account and an empty object for an unregistered address. A separate burst of ten failed logins continued to receive ordinary HTTP 401 responses, without a lockout or HTTP 429 during that sample.

Impact. The question response discloses account existence. The login sample indicates an absent or higher-than-tested threshold, not unlimited brute-force capacity. Longer-window, distributed, and account-specific protections were not established by this test.

Recommendation. Use non-enumerating recovery responses and layered login throttling. Verify thresholds over time with a dedicated test account, including recovery and reset behavior after a block.

Evidence: `ev-mtoy42l9-13o` , `ev-mtoy42tu-13p` , `ev-mtoy432j-13q` ; failed logins: `ev-mtppqb8r-3g` , `ev-mtppwknq-45` . Source entries: 45, 52.

T17 — Verbose error responses

Result: error paths disclosed implementation details. Multiple routes returned framework banners, server paths, or stack traces to the requesting client.

Test. Unknown `/rest/` paths, missing `/ftp` files, unsupported methods on tracking/configuration routes, and invalid or duplicate basket-item operations triggered verbose errors. Valid requests supplied clean baselines. The affected responses identified Express and, on relevant paths, Sequelize/SQLite components and server-side source locations.

Impact. The responses reveal deployment and code-layout information that can support further investigation. A version string or stack trace alone does not prove that a known component vulnerability is exploitable.

Recommendation. Return generic client errors with a correlation ID. Record full diagnostic information only in access-controlled server logs, and handle expected validation and uniqueness failures explicitly.

Evidence: `ev-mtojhuwm-12` , `ev-mtow4tgf-w2` , `ev-mtp2r2m0-1i5` , `ev-mtp3euon-1iw` , `ev-mtpa19zm-224` , `ev-mtpadyhd-23a` . Source entries: 41, 44, 48, 49, 50, 51.

T18 — Browser security-header gaps

Result: expected defense-in-depth headers were absent from the tested page responses. Checks on `/` did not find Content-Security-Policy, Referrer-Policy, or Permissions-Policy.

Test. Repeated page requests produced the same header omissions. HSTS was not assessed for this plain-HTTP local target. The repeated per-header entries are grouped into this deployment-hardening case.

Impact. An explicit policy would strengthen control over executable content, referrer behavior, and browser capabilities. Missing Referrer-Policy does not mean that every modern browser sends full cross-origin URLs; browser defaults still apply. Missing headers alone do not demonstrate exploitation.

Recommendation. Deploy a tested CSP, choose an explicit referrer policy, and disable unnecessary browser capabilities through Permissions-Policy. Validate HTTPS and HSTS separately in the intended production deployment.

Evidence: `ev-mtojipecf-13` , `ev-mtojiipmi-14` , `ev-mtp1l9sy-1fc` , `ev-mtp1la13-1fd` . Source entries: 42, 53, 54.

T19 — Public API data and access-policy questions

Result: data exposure observed; several original authorization claims require a policy decision.

Configuration/version, challenge/hint, feedback, and inventory endpoints returned data in contexts the initial output labeled unauthorized.

Test. Collection routes returned data without a session even where a related per-item route was guarded. Query-form object selection also returned records when the path-form request was rejected. Feedback included a wallet-seed-shaped string in a public comment.

Assessment. A route name containing `admin` , a guarded sibling route, or a nonexistent-ID control does not establish that the successful route must be private. Collection access may support public challenge and game workflows. The feedback string is sensitive-shaped content, but the test did not establish a funded live wallet or financial loss. These observations should not be promoted to separate critical/high authorization findings without establishing the intended data policy.

Next test. Define the allowed data for each role and anonymous users. Identify a specific field or action outside that policy, then repeat own/foreign/anonymous comparisons on the same resource. Apply response-field allowlists where public endpoints include unnecessary account data.

Evidence: `ev-mtojjwxb-15` , `ev-mtoketzj-3v` , `ev-mtpl0mu5-39c` , `ev-mtplrnks-1fm` , `ev-mtokwwnx-5i` , `ev-mtpuzrg4-kc` . Source entries: 3, 4, 6, 14, 27, 30, 40, 47.

T20 — Continue-code issuance and progress restore

Result: code-based progress restore observed; unauthorized forgery not established. Continue codes were available without a session and a valid code restored challenge progress.

Test. Repeated `/rest/continue-code` requests returned the same code across the tested state change. Applying a valid code without a session restored progress, while an invalid code was rejected. Attempts to apply a code from the other stream, and attempted forged variants, were rejected.

Assessment. A code acting as a bearer credential can be intentional for a training application's progress transfer. Repeatability alone does not prove an exploitable prediction algorithm, and this run did not successfully forge a code. A need for user binding depends on the intended progress-sharing policy.

Next test. Establish that policy and determine whether an attacker can derive a code for unauthorized progress without possessing a valid one. Validate any claimed cross-user or cross-stream impact before assigning a security severity.

Evidence: `ev-mtomstoa-96` , `ev-mtompcpk-91` , `ev-mtomqxj1-92` . Source entries: 28, 29, 57.

T21 — Script-bearing uploads

Result: file validation bypass observed; stored XSS not demonstrated. Upload requests were accepted when script-bearing SVG content was declared as an image type that the endpoint allowed.

Test. A truthfully declared SVG control was rejected. An SVG carrying script and declared as `image/jpeg` was accepted, and fetching the stored asset returned the submitted bytes with an image content type. Repetition established storage and retrieval, but not script execution in a browser.

Assessment. The accepted bytes show a mismatch between file contents and validation. They do not establish an executable document context. The original low-signal SVG entry and the upload-only XSS lead describe the same unresolved execution condition.

Next test. Exercise the actual viewing and embedding paths in a browser and check for a controlled execution marker. Validate decoded image content, re-encode accepted images, and serve uploads with safe content types and `nosniff`.

Evidence: `ev-mtovcvhz-ux`, `ev-mtovdf2e-uy`, `ev-mtovguid-v0`, `ev-mtpztgsl-u5`. Source entries: 58, 60.

T22 — XML handling on the B2B order endpoint

Result: XXE unresolved. `POST /b2b/v2/orders` accepted requests containing XML DOCTYPE and external-entity syntax.

Test. Benign XML and XML with a file entity both received an order confirmation. Removing authentication produced HTTP 401. Changing the entity target to a nonexistent file did not produce a useful error or content difference, and no file contents or callback were observed.

Assessment. A successful order response does not establish that the body was parsed as XML or that an entity was expanded. The Docker deployment did not provide a usable XXE challenge signal, and no out-of-band confirmation was available.

Next test. First establish that the endpoint consumes the XML fields. Then test entity expansion with controlled local content or an authorized callback service. Disable external entities and DTD processing if XML parsing is required.

Evidence: `ev-mtp8tqsp-1zl`, `ev-mtp8tql1-1zk`. Source entry: 55.

T23 — Inconsistent JWT verification between route families

Result: authentication compatibility failure observed; bypass not established. A freshly issued session worked on a basket route but failed on an API detail route.

Test. `/api/Challenges/{id}` returned HTTP 401 with a crypto `DECODER routines::unsupported` error when given the fresh token. With no token, it returned a different missing-authorization error. The same token was accepted on `/rest/basket/{id}`.

Assessment. The difference indicates inconsistent token processing or key/algorithm configuration between middleware paths. It prevented a conclusive access-control test on the affected detail route. It does not show that the route accepted an invalid token.

Next test. Compare verifier configuration and supported key formats across route families. Add a shared valid/invalid/expired-token matrix, then rerun the blocked authorization checks after the compatibility issue is resolved.

Evidence: `ev-mtpbzxhq-29i`, `ev-mtpbyps6-29e`, `ev-mtpc7zm3-29r`. Source entry: 59.

Remediation plan

The following order is appropriate when applying these results to a production application. Juice Shop is intentionally vulnerable; changes to the training deployment should preserve the purpose of the exercise.

Priority	Work	Acceptance check
1	Fix login/search SQL injection and unsigned-token acceptance	Boolean/UNION payloads fail; unsigned, invalid, and expired tokens are rejected
1	Enforce ownership and role checks on address, basket, recycling, catalog, and order routes	Own, foreign, and anonymous tests follow the same defined policy for every HTTP method
1	Remove credential fields and sensitive files from public responses/storage	API schema tests exclude secrets; direct and encoded backup paths are denied
2	Correct DOM rendering, input validation, and redirect validation	No browser execution marker; invalid quantities/amounts/ratings fail; external destinations are rejected
2	Strengthen authentication abuse controls and error handling	Recovery responses do not enumerate users; configured throttles trigger; errors expose no stack traces
3	Resolve policy and execution questions in T19–T23	Each follow-up has either a repeatable unauthorized effect or a documented intended behavior
3	Add browser security headers and retest the intended HTTPS deployment	Header policies work without breaking expected application flows

Result traceability

The source export contained 60 entries after its automated duplicate corrections. The mapping below accounts for every entry. Entry numbers refer to that reviewed export, not to the new case IDs. Evidence identifiers in each case point to the saved requests or browser records used by the assessment.

Case	Reviewed-export entries	Subject
T01	2	Login SQL injection
T02	1, 46	Search SQL injection and hidden-product retrieval
T03	26	Search-route DOM XSS
T04	23	Unsigned JWT acceptance
T05	5, 12	Basket and basket-item reads
T06	24	Address write and ownership transfer
T07	10, 11	Recycling reads and ownership assignment
T08	18, 19, 20	Catalog writes and metadata assignment
T09	15, 16, 17	Tracking data and invoices
T10	9, 13, 32, 34, 35	Account-field and digest exposure
T11	8, 21, 25, 33, 37, 39, 56	File exposure and extension bypass
T12	22	Negative basket quantity
T13	36	Negative wallet deposit
T14	7, 31, 43	Feedback controls
T15	38	Redirect allowlist bypass
T16	45, 52	Enumeration and login-burst behavior
T17	41, 44, 48, 49, 50, 51	Error disclosure
T18	42, 53, 54	Browser security headers
T19	3, 4, 6, 14, 27, 30, 40, 47	Public API policy and exposed content
T20	28, 29, 57	Continue-code workflow
T21	58, 60	Upload validation and unproven execution
T22	55	Unresolved XML entity processing
T23	59	JWT verifier compatibility

Report handling

This report describes the saved September assessment; preparing it did not initiate another target scan. Credentials and personal values are excluded, and the public package does not include raw session or traffic files. Test object IDs and evidence references belong to that local run; use dedicated accounts and objects when reproducing the tests in another authorized environment.

[Assessment record and supporting cost image](#) · [HTML report](#) · [PDF report](#)